

5CM510 – Network System development

Lecture 15: Network Function Virtualization

Lecture Outline

- Network function virtualization (NFV) and Virtual network functions (VNFs)
- Modelling Network functions NFs and VNF Placement problem.
- Summary

Section 1

NFVs and VNFs: Definitions, Scope, architecture, Use cases and examples, challenges, orchestration and Performance

NFV and VNF: Definitions

- Network function virtualization (NFV) represents a significant transformation for telecommunications/service provider networks, driven by the goals of reducing cost, increasing flexibility, and providing personalized services . It is rapidly emerging as the de facto approach operators will use to deploy their networks.
- NFV leverages on cloud computing principles to change the way Network functions (NFs) such as gateways and middleboxes are offered.
- Different from today's tight coupling between the network functions (NF) software and dedicated hardware, the loosely coupled software and hardware in NFV can reduce the upgrade cost and increase the innovation flexibility.
- Typical NFs (e.g., server load balancing (SLB), and firewalling) in large-scale enterprise networks today are generally provided on large vendor-proprietary devices.
- These network functions (NFs) include both control plane processing, such as telco equipment for signalling, authentication, and data plane processing, such as normal routing, access control, and quality of service (QoS). These NFs run as software in a virtual machine (VM), which is also called virtualized network function (VNF).

The scope of NFV

- NFV covers a wide spectrum of middleboxes such as firewalls, deep packet inspection (DPI), intrusion detection system (IDS), network address translation (NAT), and wide area network (WAN) accelerators.
- It also covers a variety of network nodes such as broadband remote access servers' data network gateways (S-GW/P-GW), mobility management entity (MME), home subscriber server (HSS), and virtual IP multimedia subsystem (vIMS) for virtual evolved packet core (vEPC). These are the critical devices in the mobile broadband and cellular networks.
- VNFs range from traditional telecom devices to security appliances. In the trend of microservices, the VNFs will become small and modular.
- New services can be built by composing multiple microservices together in a flexible way.

NFV architecture

- The concept of NFV was initiated within the European Telecommunications Standards Institute (ETSI) consortium.
- NFV allows legacy NFs offered by specialized equipment to run in software on generic hardware. The key driving technologies behind that is cloud computing and virtualisation.
- NFV makes it possible to deploy VNFs in high-performance commodity servers in an operator's data center, with great flexibility to spin on/off the VNFs on demand.
- By decoupling the NF software from the hardware, NFV will facilitate a faster pace for innovations and result in shorter service development cycles. In summary, NFV has great potential in CAPEX/OPEX savings and ARPU increasing.
- The objective of NFV is to separate software that defines the VNF from hardware and generic software that creates a generic hosting network functions virtualization infrastructure (NFVI), which executes the VNF.
- The VNFs and the NFVI are then separated from each other.

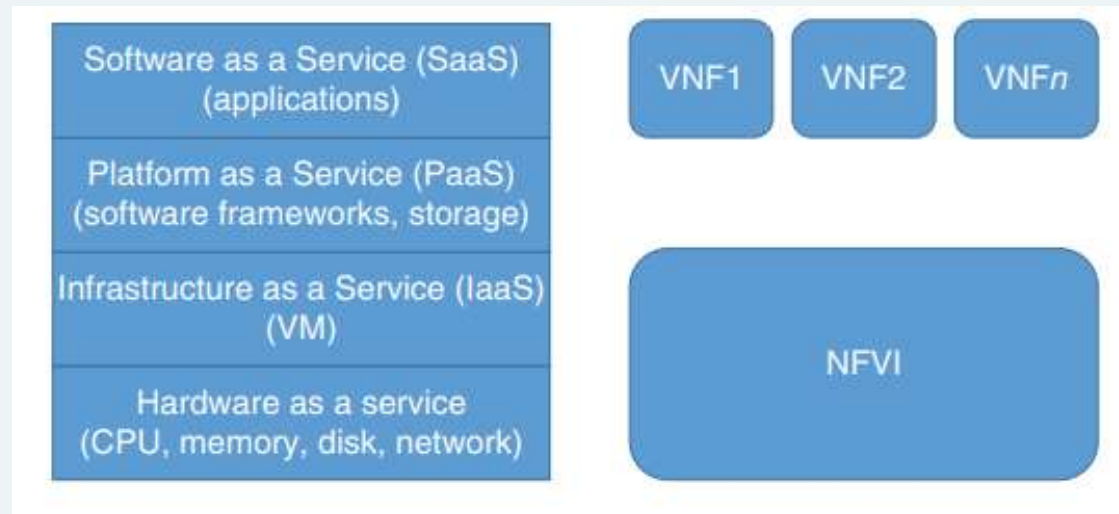
Types of VNFs and Examples

Edge devices	vCPE, IP Edge, BRAS
Gateway functions	IPsec, SSL, NAT
NGN signaling	IMS, VoIP, HSS
Application optimization	CDN, cache, video transcoder, HTTP header enrichment
Performance improvements	Load balancer
Service assurance	SLA monitoring, testing
Mobile network equipments	HLR, SBC, MME, SGSN, GGSN, RNC, eNodeB
Security functions	DPI, Anomaly detection, Firewall, IDS, IPS

Technologies of Clouds in NFVI

- NFV will also have broad applicability in large enterprise networks to reduce their CAPEX and OPEX.
- We discuss four key technologies of cloud computing that are used in NFVI.
- On-demand, self-service: operators can easily allocate resources such as server time, storage, and network.
- Flexible and board network access: any VNF can route traffic to another VNF freely if needed.
- Resource pooling: Within a single NFVI point of presence (PoP), one type of VNF may need to deploy on multiple instances. To support dynamically changing resource allocation, VNFs can be deployed on a pool of available resources.
- Rapid elasticity for the sake of load surge, failover, and power optimization.

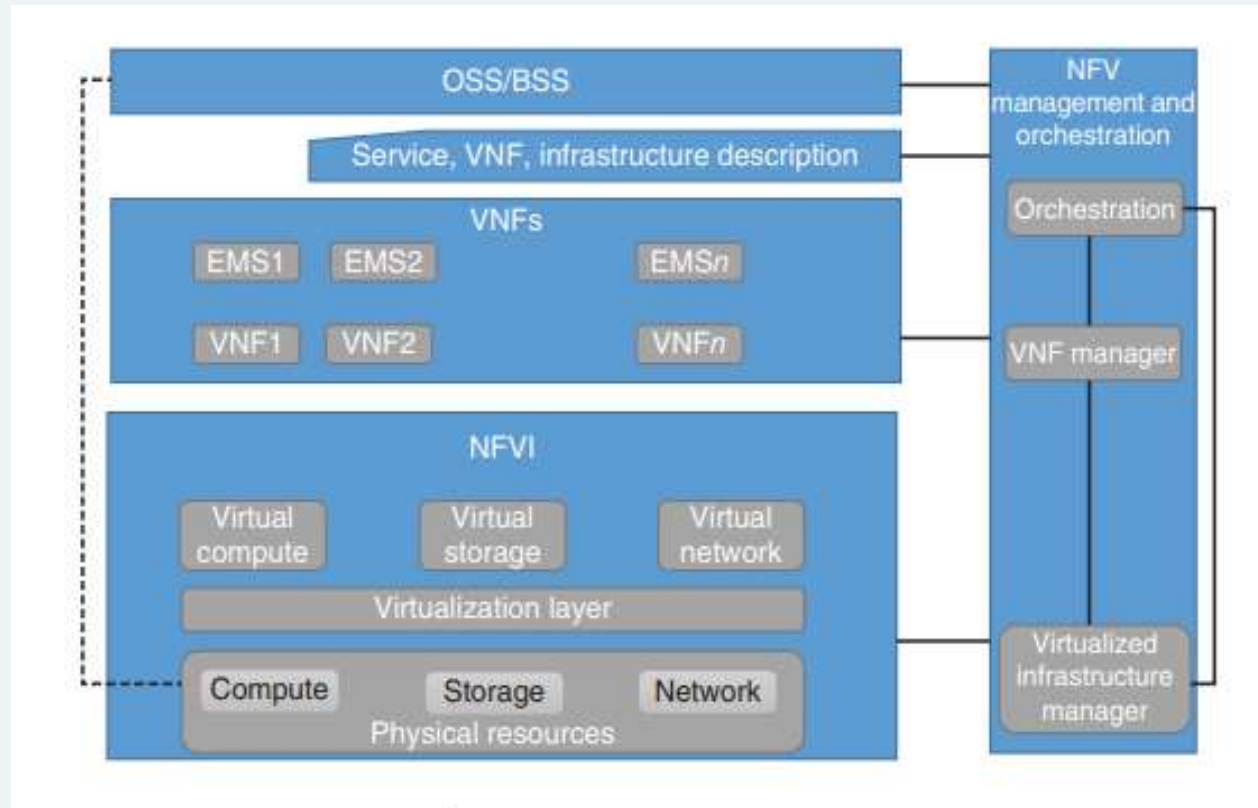
NFV Architecture in Cloud computing stacks



ETSI NFV architecture

- NFVI: The NFVI provides the technology platform with a common execution platform for a variety of VNFs. It consists of a distributed set of NFVI nodes in various locations to support the locality and latency requirements of the different use cases. NFVI can be categorized into three domains: the compute and storage domain, the hypervisor domain, and the infrastructure network domain.
- NFV orchestrator: The objective of NFV is to separate the VNFs from the infrastructure, and this includes their management. This management and orchestration block is often complex and composed of great many distributed component parts.
- VNF manager(s): It is responsible for VNF life cycle management, including VNF creation, resource allocation, migration, and termination.
- Virtualized infrastructure manager(s): It is responsible for controlling and managing the computing, storage, and network resources, as well as their virtualization.
- OSS: Similar to OSS in traditional cellular network infrastructure, the main role of OSS in NFV is supporting the comprehensive management and operations functions.
- EMS: It performs the typical FCAPS functionality for a VNF. The current element manager functionality needs to be adapted for the virtualized environment.

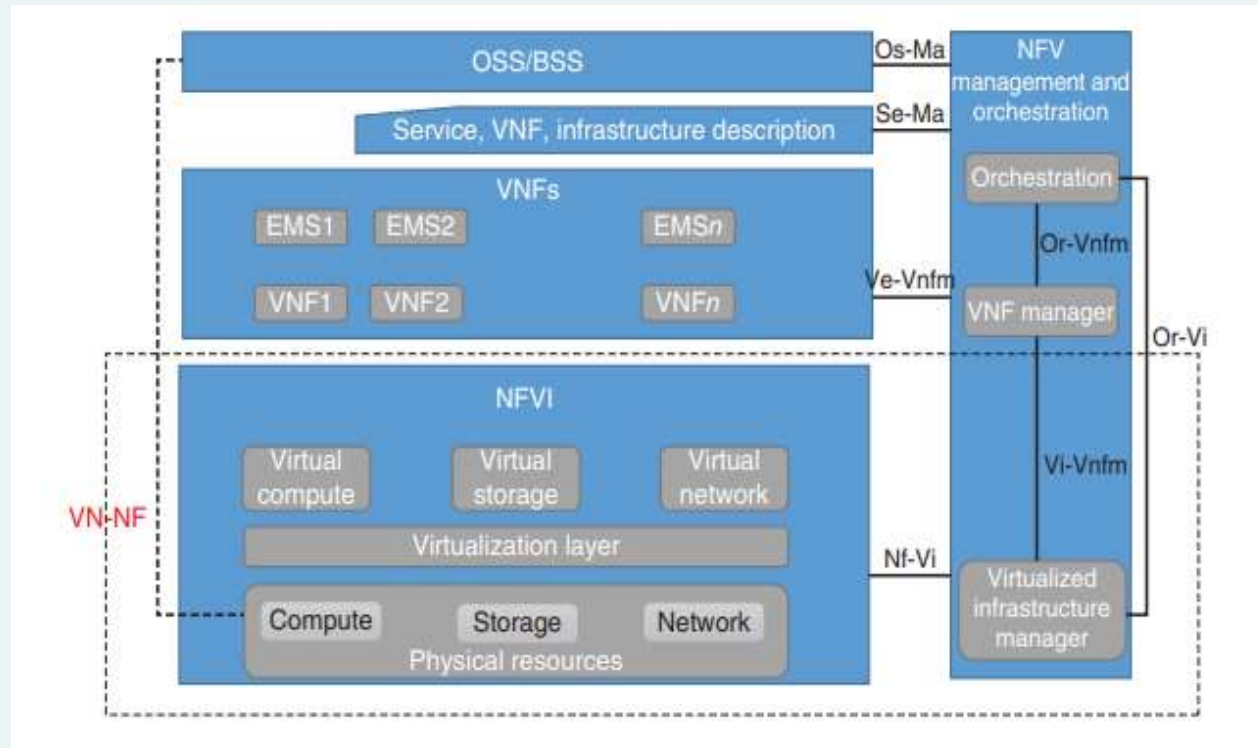
ETSI NFV Reference Architecture



OPNFV architecture

- The open-source community OPNFV has been growing and pushing the open-source development of the NFV infrastructure. One open-source NFV activity is called OPNFV [13].
- The OPNFV architecture provides the best carrier-grade NFVI, virtualized infrastructure management (VIM), and APIs to other NFV elements.
- It forms the basic infrastructure required for VNFs and Management and orchestration (MANO) components.

OPNFV Architecture



ETSI's 9 Use cases and Examples for NFV

- **Among all these use cases, 1–4 are in the cloud computing area, 5 and 6 are in mobile computing, 7 is for data center, and the last two are in residential and access networks.**
- NFV Infrastructure as a Service (IaaS) : run NFs on the infrastructure that belong to service providers.
- Virtual network function as a service: purchase functions whenever and wherever needed as a pay-as-you-use model.
- Virtual network Platform as a Service (PaaS): With NFV, the virtual network platform includes not only the routing and switching gear but also various network functions, which can provide additional security and QoS to the enterprise network.
- VNF forwarding graphs: pipeline of VNFs that traffic should traverse through. It is also known as service function chain (SFC).
- Virtualization of mobile core network and IMS: Virtualizing the core network facilities such as Evolved packet core (EPC) or IP multimedia subsystem (IMS) can increase the capacity of a mobile core network dynamically.

ETSI's 9 Use cases and Examples for NFV

- Virtualization of mobile base stations: virtualisation of radio access network. After virtualizing base stations, multiple service providers can share the same physical resource so that the coverage and resource utilization are both increased.
- Virtualization of home environment: The residential gateway or home gateway is another place where traditional network services are deployed as dedicated hardware. When upgrading the home gateway, often a new hardware needs to be shipped together with manual installation. Typical network services in a home environment include parental control, DPI, firewall, and so on. With NFV, the gateway can simply be a forwarding device while other complex functions can be run in the providers' data center. This can help reduce the operational overhead and ease the management tasks.
- Virtualization of CDN: Virtualizing the CDN nodes and deploy them in the cloud can provide resource elasticity as demand changes.
- Fixed access network functions virtualization: The main fixed line access network technologies today are based on digital subscriber line (DSL), for example, VDSL or FTTdp. Access network functions virtualization can greatly simplify these remote devices.

NFV Challenges

- Carrier-grade properties: The telecom service providers have high requirement on the performance, scalability, fault tolerance, and security on the solutions.
 - Efficiency: The NFV platform should provide the tight NF SLAs on performance or availability, identical to the SLAs offered with dedicated services.
 - Scalability: The platform should support many VNFs and scale as the number of subscribers/applications/traffic volume grow.
 - Reliability: The platform should abide by NFV reliability requirements. Service availability, as defined by NFV, refers to the end-to-end service availability that includes all the elements in the end-to-end service (VNFs and infrastructure components).
- Elasticity: Building on top of the virtualization technology, an NFV platform should be able to leverage the benefit of running instances in the cloud: multiplexing and dynamical scaling. For multiplexing, it allows the same NF instance to serve multiple end users to maximize the resource utilization of the NF.
- Openness: Aligned with the Open NFV strategy in the HP NFV business unit, the NFV framework should be capable of accommodating a wide range of NFs in a nonintrusive manner. It should support open-source-based and standard solutions as much as possible, meaning that the NFs should be implemented, deployed, and managed by operators, enterprises, or third-party software vendors.

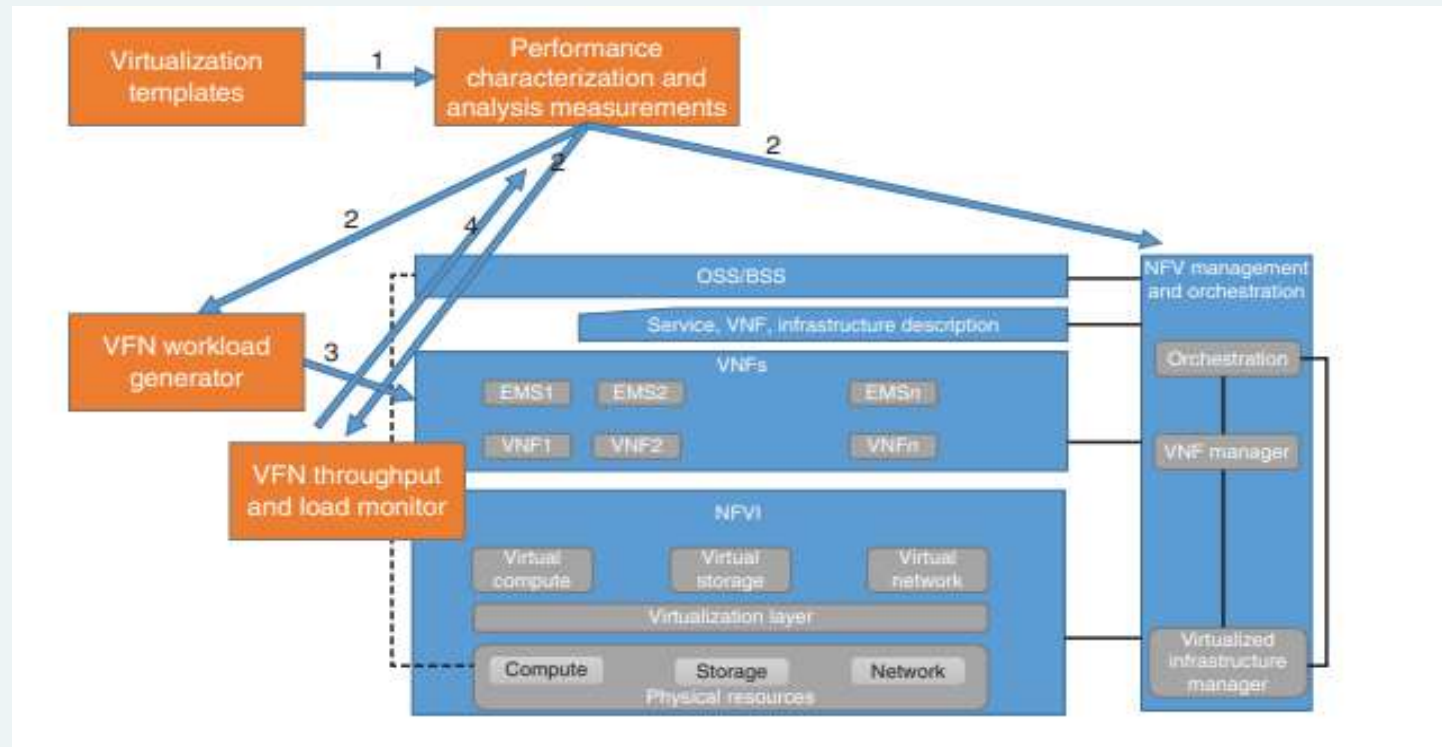
NFV Orchestration - MANO

- NFV management and orchestration component, that is, MANO, is an integral piece of NFV architecture and a critical component to the success of NFV. It is responsible in managing all the resources for NFV framework; not just the networking resource but also the compute and storage resources.
- According to ETSI's MANO working group, it contains three key components: the NFV orchestrator, VNF manager, and virtualised infrastructure manager (VIM).
- **NFV orchestrator** is responsible for the installation and preparation for new services, which is also called the network service onboarding process.
- **VNF manager**, as the name indicates, handles the life cycle of each VNF instance. It includes the configuration, preparation, and running of the VNF instance, monitoring its healthiness and interacting with other MANO components on behalf of the VNFs.
- **VIM** controls and manages the NFVI components, to better accommodate the need for various network services. For compute resource, it manages the physical resources and the virtual resources, for example, servers, VMs, CPUs, and memories.

NFV Performance

- Benchmarking the VNF's network and compute performance in real NFV deployment is important for resource allocation.
- A typical VNF performance characterization process, the testing is performed for each configuration.
- Different virtualization technologies, such as Intel DPDK, SR-IOV, and VT-d, are configured depending on the scenarios.
- Tests are done with different VM placement and configurations. VMs may be deployed on the same hosts or across hosts, both have different impact on the real performance.

VNF Performance Characterization



NFV Performance improvements

- Monolithic complex network functions running on specialized hardware are decomposed into smaller functional units and dynamically orchestrated onto a virtualized cloud and edge infrastructure.
- These network functions include both control plane processing (signalling, authentication, etc.) and data plane processing (routing, firewalling, etc.) and are encapsulated in a VM as a VNF.
- Service chaining defines routing instructions that direct network packets through a pipeline of VNFs. A core component of the NFV infrastructure is a highly agile network that enables efficient and dynamic service chaining.
- While the NFV transformation is under way, another trend is the move toward using large pools of compact, power-efficient compute nodes/SoCs that are integrated with large stores of fast persistent memory and custom high-speed interconnects

Section 2

White box and black box NF Models, VNF Placement Problem

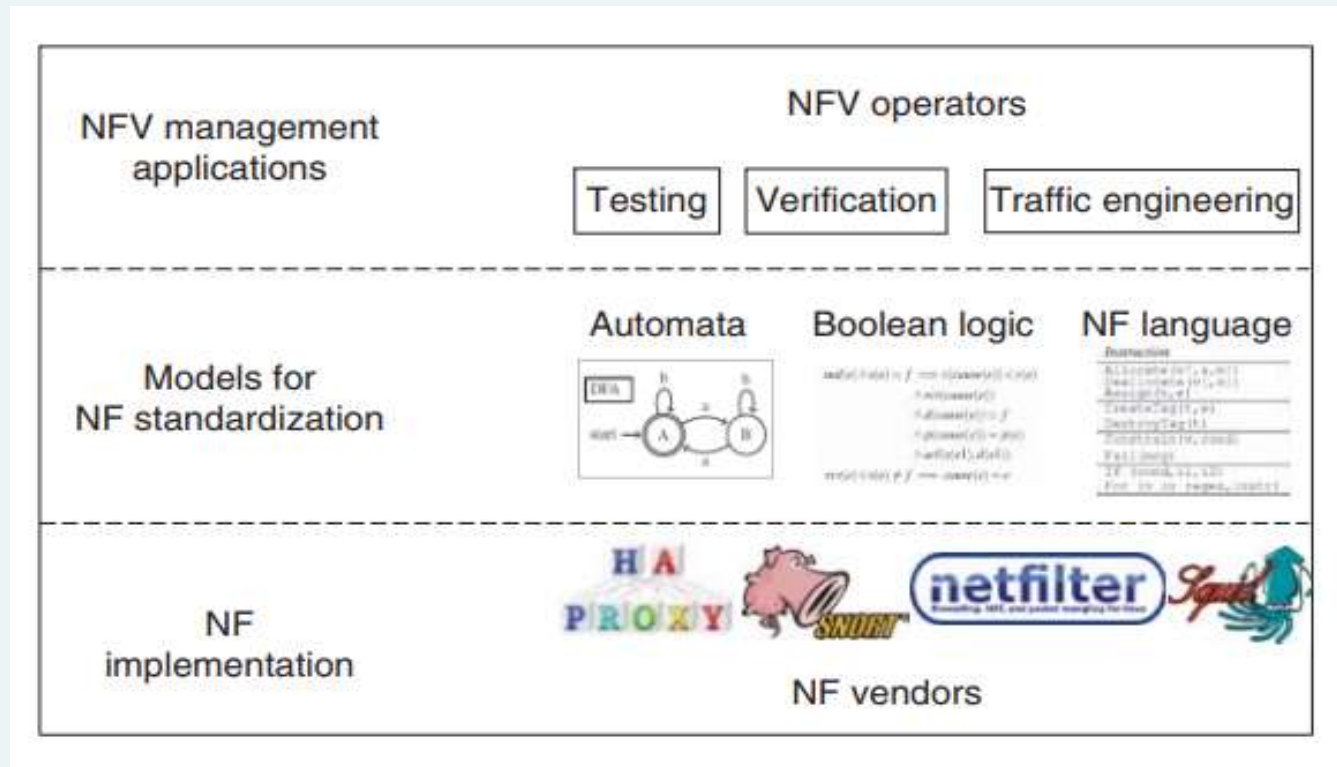
Models of NFs

- With the deployment of proprietary NFs, many network management functions (e.g., testing, policy compliance, and vendor interoperability) need accurate models of NFs to ensure correctness and reliability. However, due to their proprietary and complex nature, there is a lack of standardised behaviour model of these NFs. There are representative modelling approaches:
- **High-level box view:** In most existing NFV platforms and NF performance optimization solutions, an NF is viewed as a black box with traffic traversing it, but the NF's internal logic is not differentiated. In practice, an NF is implemented as a VM with the NF software running inside it, the VM is deployed on general servers, allowing traffic to be routed through.
- **Manually generated models:** A set of works view NFs as deterministic finite automata (DFA). The DFAs in these current models are manually generated and directly used to replace the NFs, which implies possible incompleteness.
- **Modelling language design:** symbolic execution-friendly language to remove these two features for NF programming. Both works aim to develop NF programs that can be symbolic executed so that program bugs can be found out early during the development.

NFV Ecosystem and NFV models

- The important role that NF models play in the whole NFV ecosystem. NF models connect NFV applications and NF implementation.
- On the one hand, from NFV operators' view, several applications on top of NFV platforms are based on NF models.
- For example, when an operator needs to generate test traces for deployed NFs, the operator needs to assume NFs' behaviors (i.e., model) to completely cover the test space; the operator verifies network-wide properties (e.g., reachability), whether a flow gets through an NF needs to be inferred from the NF model.
- Existing works usually build NF models according to their application scenarios, so that the models are tailored specific to their applications and not generic.
- These fall into two categories. The first category aims to build models used for network management tasks (e.g., testing, traffic engineering, and migration), and the second one focuses on improving NF software, including its performance, reliability, and robustness.

NF Modelling



White box NF

- Analyses the source code of a given network function to automatically generate an abstract packet forwarding model. It is built on top of the following code analysis methods.
- Programme slicing: A program slice is a minimum set of statements in a given program that lead to certain behaviors. The behaviour is expressed as the values of variables in a certain statement.
- The basic methodology to get a program slice is to analyse the dependency between program statements.
- Symbolic execution: A program analysis approach that substitutes one or several program variables by symbolic values. As the program runs, whenever a branch instruction is met, the program is forked and both branches proceed.
- Symbolic execution can exercise all possible execution paths, but path explosion can happen with a large code base. Various efforts (e.g. reduce the branching factor) have made symbolic execution practical in NF verification.

NF State Analysis

- NF State analysis: OpenNF is a flow state management framework that enables joint control with Software defined networking (SDN) and NFV frameworks. StateAlyzer defines features of state variables in an NF program, leveraging several program analysis techniques (e.g., program slicing, system dependency analysis) to identify them.
- These features and categorization of variables are used for code analysis to generate the NF:
- Persistent: The variable has a lifetime longer than the packet processing loop.
- Top level: The variable is used during packet processing.
- Updateable: The variable's value is updated during packet processing, that is, usually an LHS variable.
- Output-impacting: The variable impact variables in the packet output function.

NF Modelling: Layer 4 balancer code and a slice

```

1 from scapy.all import *
2 # Constants
3 ROUNDROBIN = 1
4 MIU = 1500
5 # Configurations
6 mode = ROUNDROBIN
7 LB_IFACE = "eth0"
8 LB_IP, LB_PORT = "3.3.3.3", 80
9 servers=[("1.1.1.1", 80), ("2.2.2.2", 80)]
10 # Output-Impacting States
11 f2b_nat, b2f_nat={}, {}
12 rr_idx = 0
13 cur_port = 10000
14 # Log States
15 pass_stat, drop_stat=0, 0
16 # callback function
17 def pkt_callback(pkt):
18     global drop_stat, pass_stat, rr_idx, cur_port
19     si, di = pkt[IP].src, pkt[IP].dst
20     sp, dp = pkt[TCP].sport, pkt[TCP].dport
21     if dp == LB_PORT: # pkt from client to server
22         cs_ftpl, sc_ftpl = (si, sp, di, dp), (di, dp, si, sp)
23         if cs_ftpl not in f2b_nat: # new connection
24             if mode == ROUNDROBIN:
25                 server = servers[rr_idx]
26                 rr_idx = (rr_idx+1) % len(servers)
27             else: # Hash to a backend server
28                 server = servers[hash(si) % len(servers)]
29             n_port = cur_port
30             cur_port+=1
31             cs_btpl = (LB_IP, n_port, server[0], server[1])
32             sc_btpl = (server[0], server[1], LB_IP, n_port)
33             f2b_nat[cs_ftpl], b2f_nat[sc_btpl]=cs_btpl, sc_ftpl
34             nat_tpl = cs_btpl
35         else: # existing connection
36             nat_tpl = f2b_nat[cs_ftpl]
37         else: # pkt from server to client
38             sc_btpl = (si, sp, di, dp)
39             if sc_btpl in b2f_nat:
40                 nat_tpl = b2f_nat[sc_btpl]
41             else: # no initial outbound traffic is allowed
42                 drop_stat+=1
43                 return
44             pass_stat+=1
45             pkt[IP].src, pkt[TCP].sport = nat_tpl[0], nat_tpl[1]
46             pkt[IP].dst, pkt[TCP].dport = nat_tpl[2], nat_tpl[3]
47             for f in fragment(pkt[IP], fragsize=MIU-len(Ether())):
48                 sendp(Ether()/f, iface=LB_IFACE)
49 def LoadBalancer():
50     sniff(iface=LB_IFACE, prn=pkt_callback, filter="tcp")
51 if __name__=="__main__":
52     LoadBalancer()

```

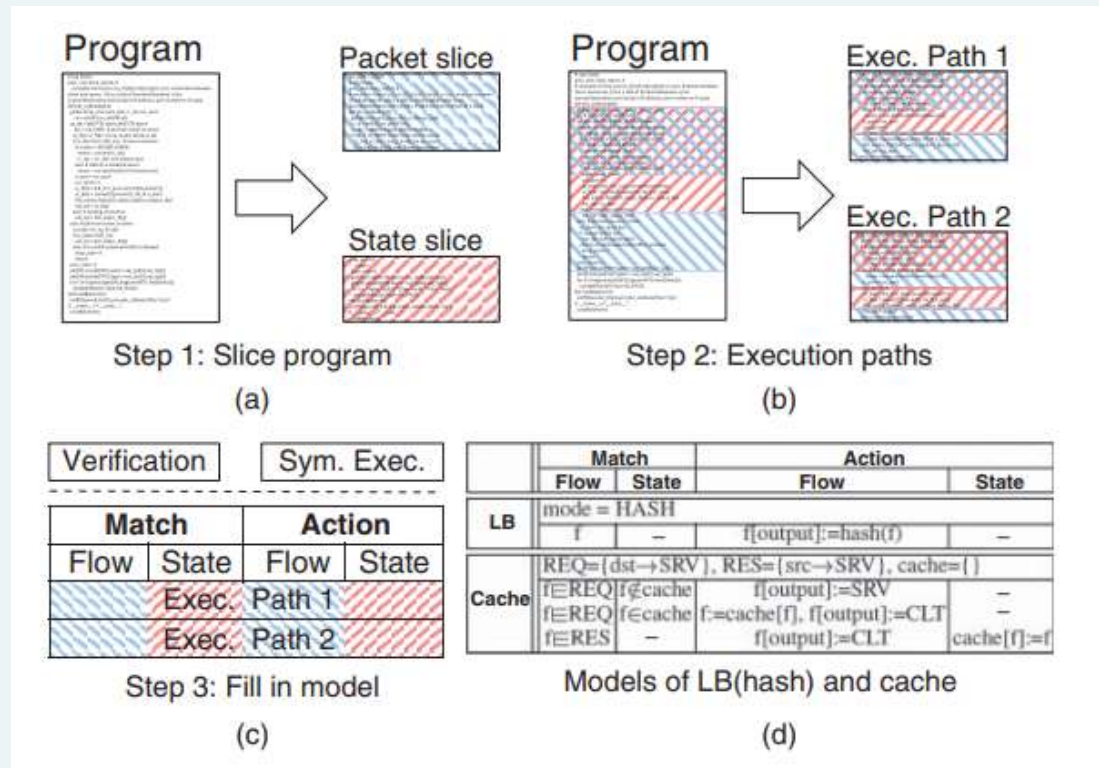
*variable format (cs—sc)_(f—b)(s—d)(i—p—tpl): sc—cs is direction between client and server, f—b is a side of frontend/backend, s—d is source/destination and i—p—tpl is IP, port or 4-tuple.

Match		Action	
Flow	State	Flow	State
Configuration = C_1			
f_1	$s_1 \wedge P(f_1, s_1)$	$Fwd(f_1, s_1)$	$Upd(f_1, s_1)$
...	...	...	...
Configuration = C_2			
f_2	$s_2 \wedge P(f_2, s_2)$	$Fwd(f_2, s_2)$	$Upd(f_2, s_2)$

Model Extraction Overview

- A NF program is assumed to have a processing loop and packet input/output functions, and an NF model is predefined like a stateful match action table. This takes three steps as follows:
- 1) It first conducts backward slicing from packet output function to get packet slice and from state variable assignment statements to get state slice.
- 2) It then symbolically executes the union of both slices and get multiple execution paths. For scalability, we need to constrain the value of some variables.
- 3) Finally, each path is filled into the stateful match action table as a row, conditional statements about packets/states are put into flow/state fields of match column, and intersection of an execution path with packet/state slice is put into flow/state fields of action column.

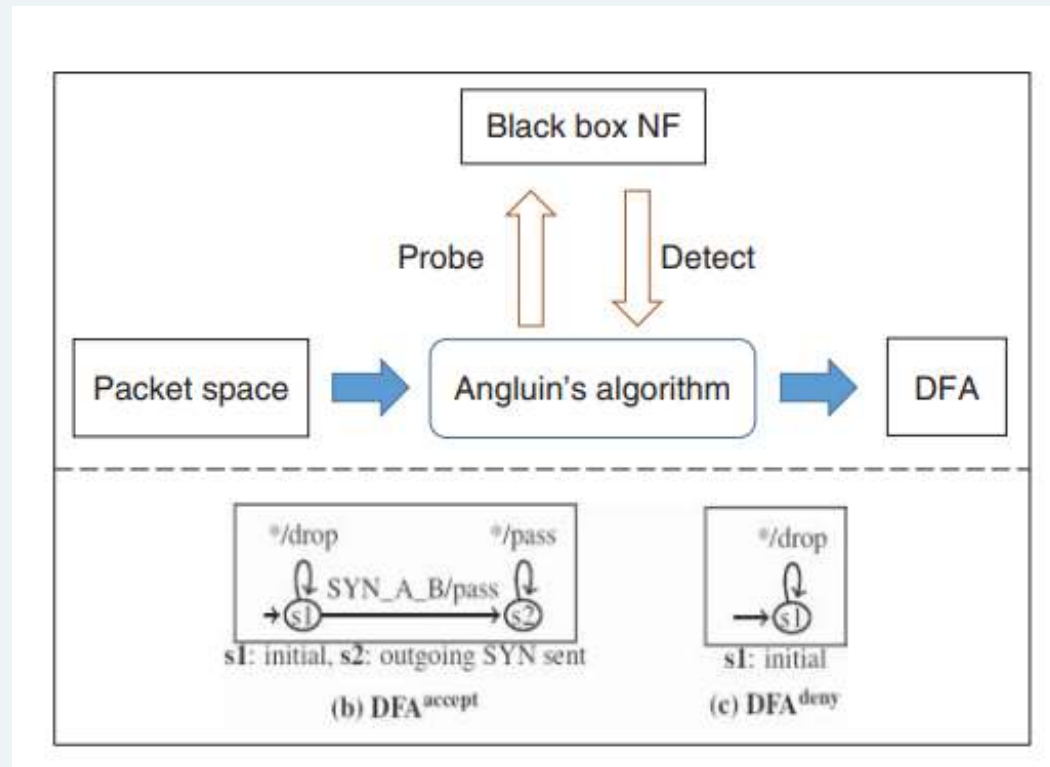
Source-code based NF Modelling method



Black box NF

- Black box synthesis of NF models, if viable, can faithfully match the observed input/output behaviour without needing access to the source code. It has several natural advantages:
- (1) the approach is faithful to input/output behaviour (e.g., states need to be manifested in handling traffic) and
- (2) It does not rely on having the source code of NFs, and thus operators can extract models without vendor support.
- Formulate the black box NF model synthesis as an “active learning” problem of inferring the FSM model that represents the NF behaviour. A whole NF instance can be viewed as a black box and probe its internal logic by sending packets/requests to it and observing its reactions.
- A system that takes the packet space (filtered by related header fields) as input, interacts with the black box NF, and leverage an active learning algorithm proposed by Dana Angluin to generate a DFA model.
- The main idea of Angluin’s algorithm is to iteratively assume a DFA model, compare the DFA with the NF, and refine the DFA until no difference is found between the DFA and the NF.

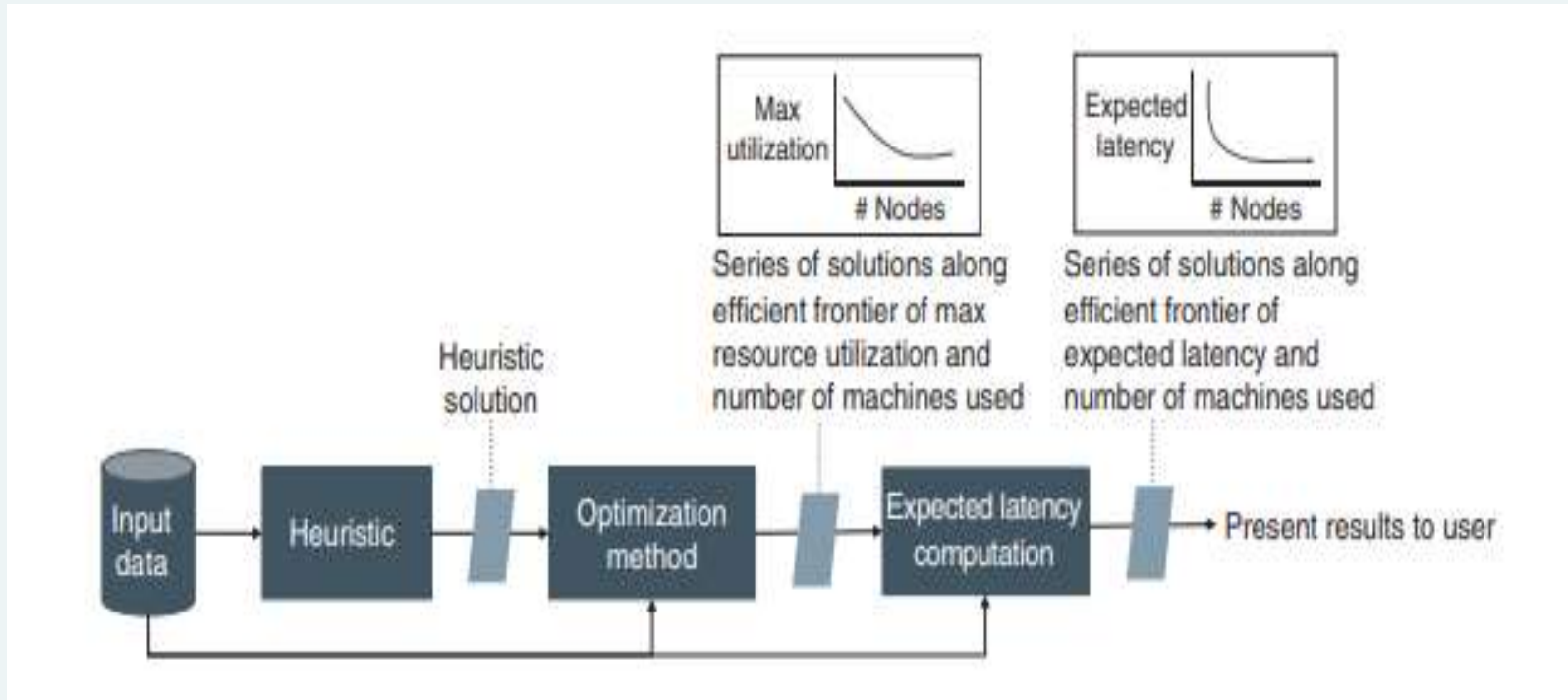
Black Box NF Modelling method



VNF Placement problem - Optimisation

- VNF placement problem entails choosing where to place instances of VNFs on servers in a physical network to accommodate the traffic for a set of service chains.
- Examples are firewall services, intrusion detection, caches, and proxies.
- Each service chain is a stream of network packets flowing through a sequence of VNFs at a certain rate. Network traffic for a given service chain must visit the chain's sequence of VNFs in the specified order.
- The VNF placement problem resembles that of virtual network embedding (VNE), in which one must determine how to place a set of virtual networks onto a physical substrate network.
- In one respect, each service chain in a VNF placement problem can be viewed as a virtual network to be mapped onto a physical network.
- VNF placement differs in that each node (VNF) in a virtual network can be mapped to multiple instances that are placed on different nodes in the physical network. VNE approaches do not address how to map each VNF into instances.

VNF Placement



Summary

- We introduce the NFV and VNF and how they relate to each other.
- We first present the NFV architecture proposed by ETSI and its relationship with cloud computing.
- NFV use cases and examples.
- Reduced equipment costs and reduced power consumption through consolidating equipment and exploiting the economies of scale of the IT industry.
- Increased speed of time to market by minimizing the typical network operator cycle of innovation.
- Availability of network appliance multi-version and multitenancy, which allows the use of a single platform for different applications, users, and tenants.
- Targeted service introduction based on geography or customer sets is possible. Services can be rapidly scaled up/down as required.
- Enables a wide variety of ecosystems and encourages openness.
- We also discussed the challenges in virtualizing the NFs and using them in telecom network.



THANK YOU